Specifying Token-Cost as a Trustworthy-AI Requirement: An Empirical Study of Silent Placeholder Prompting Across Five Small Open-Source LLMs

Venkat Chavan Nagabhushana
MSc Computer Science: Big Data and Artificial Intelligence
venkatchavan16@gmail.com

***Abstract*—Cost is a requirement. When an organisation deploys a large language model (LLM) to reason over user inputs, the bill scales with the number of output tokens, and so do latency, energy, and the size of the audit trail that a trustworthy-AI process is expected to keep. Chain-of-thought (CoT) prompting, the dominant accuracy recipe, makes that trail long on purpose; for many real systems, that is exactly the property a requirements engineer must trade off against budget, sustainability, and response-time targets. We study a lightweight prompting condition we call *silent placeholder prompting* (SPP), in which a small number of inert placeholder tokens sit between the question and the answer slot and the model is told not to verbalise its reasoning. SPP is purely a prompt-level intervention: no fine-tuning, no tokenizer change, no architectural access. We evaluate it as one *requirement-aware* option in the design space, against direct answering, full CoT, length-controlled short CoT, and a hybrid baseline, across five open-source instruction-tuned models (0.5B–3.8B parameters) and five reasoning benchmarks (GSM8K, SVAMP, ARC-Challenge, StrategyQA, PrOntoQA), in a 13,520-call evaluation matrix run on commodity hardware. On our sample, full CoT reaches the highest mean accuracy (47.4%) but spends 236.8 output tokens per response; SPP comes in at 40.6% accuracy for 15.4 tokens, a 93.5% reduction in tokens for a 6.8 percentage-point accuracy loss. The trade-off is not symmetric across tasks: SPP matches or beats CoT on classification-style benchmarks (ARC-C, StrategyQA, PrOntoQA) and loses heavily on arithmetic word problems (GSM8K, SVAMP). We then discuss what this means for requirements engineering: which trustworthy-AI properties (transparency, accountability, sustainability, cost-equity) tighten or loosen as a deployment slides along the SPP–CoT axis, and what kinds of acceptance criteria a practitioner would write down to make the trade-off auditable. We are explicit that $N{=}20$ per cell with a single seed is preliminary, and we provide an expanded evaluation plan, including bootstrap CIs and stronger baselines. The contribution is empirical and methodological, not mechanistic; we make no claim that the model performs hidden computation behind the placeholder tokens.**

***Index Terms*—trustworthy AI, requirements engineering, large language models, chain-of-thought prompting, token efficiency, cost-aware evaluation, sustainability, auditability, empirical study.**

I. Introduction

The conversation around trustworthy AI has, for understandable reasons, been dominated by safety and fairness. But anyone who has tried to put an LLM-based system into production knows the picture is messier than that. A typical procurement document for an AI service ends up listing an uncomfortable mix: response time targets, monthly token budgets, audit log retention, explainability obligations, regulatory commitments such as the EU AI Act [1], and an internal sustainability constraint that nobody wants to be the first to break [2], [3]. Some of these are "soft" organisational policies; others are hard, contractual numbers. Either way, they are *requirements*, and they interact.

Chain-of-thought prompting [4], [5] sits right at the centre of those interactions. When a model is asked to think step by step, accuracy improves on multi-step problems, but the response becomes long — often by a factor of ten or more compared with a direct answer. Long responses cost money, take longer to deliver, draw more energy, and generate more tokens that an audit pipeline has to retain and (sometimes) review. Crucially, a longer rationale is not always a *more useful* rationale: prior work has shown that the verbalised chain is not always the model's actual decision process [6], [7], which is awkward when an auditor is reading it as if it were. So the requirements engineer is faced with a question that is rarely framed cleanly in the literature: *how should I specify the verbosity of model reasoning, given that verbosity is correlated with accuracy, cost, latency, sustainability, and the perceived (but not always real) auditability of the system?*

This paper looks at one concrete option in that design space. We study *silent placeholder prompting* (SPP), in which the prompt contains k inert placeholder tokens (literally the strings `<latent_1>`, ..., `<latent_k>`) between the question and the answer slot, with an instruction not to verbalise reasoning and not to echo the placeholders. SPP is a prompting condition, nothing more. There is no fine-tuning, no special tokenizer entry, no architectural hook. It is conceptually inspired by the pause-token line of work [7], [8] but, unlike that line of work, it can be deployed against any unmodified instruction-tuned model. That makes it cheap to specify, cheap to run, and easy to roll back — three properties a requirements engineer cares about when picking a default.

We position the paper through three lenses, in this order:

1) **An RE-for-trustworthy-AI lens.** We treat output verbosity not as a hyperparameter but as a requirement that has to be specified alongside accuracy, latency, and audit obligations. We sketch how practitioners might write acceptance criteria for SPP-style and CoT-style configurations, and where SPP enables (or breaks) other trustworthy-AI properties such as transparency and sustainability.
2) **An empirical lens.** We run a 13,520-call evaluation matrix over five small open-source models (0.5B to 3.8B parameters) and five reasoning benchmarks, and characterise the accuracy–cost trade-off of SPP against direct answering, full CoT, short CoT, and a hybrid baseline.
3) **A methodological lens.** We are upfront about what the current evaluation does not yet support. With $N{=}20$ per cell and a single seed, the headline numbers are point estimates. We document this, and we ship an expanded evaluation plan (more samples, multiple seeds, bootstrap confidence intervals, stronger baselines) so the work can be re-used as a baseline rather than read as a final claim.

Research questions. We organise the empirical study around two RE-flavoured questions:

- **RQ1 (cost):** How does SPP compare with CoT, short CoT, direct, and a hybrid baseline along the four cost axes that show up most often in trustworthy-AI specifications: output tokens, latency, tokens-per-correct-answer, and accuracy-per-token?
- **RQ2 (task-conditional accuracy):** Does the accuracy gap between SPP and CoT depend on the task type, in a way that a requirements engineer could exploit by routing different request classes to different prompting conditions?

Contributions. (i) A reframing of token-efficient prompting as a trustworthy-AI *requirement specification* problem, with a small catalogue of acceptance criteria. (ii) A cross-model, cross-dataset, prompt-only empirical comparison of SPP against four baselines on a 13,520-call matrix, with all real numbers reported and figures released. (iii) A characterisation of when SPP wins (classification-like multiple choice, yes/no implicit reasoning) and when it loses (arithmetic word problems), with a failure-mode taxonomy. (iv) A frank limitations section and an expanded evaluation plan with bootstrap CIs, paired tests, and four stronger baselines (self consistency, least-to-most, tree-of-thought style, compressed rationale). We do not claim a new reasoning mechanism; the value is in the requirement-aware framing and the reproducible characterisation.

II. Background and Related Work

A. Requirements Engineering for Trustworthy AI

A growing body of work argues that traditional requirements engineering is necessary, but not sufficient, for AI-based systems [9], [10]. Two strands matter for us. The first is the regulatory strand: the EU AI Act [1] and the NIST AI RMF [11] both push practitioners to write down concrete, testable trustworthy-AI properties (transparency, accountability, robustness, fairness) rather than treat them as ambient virtues. The second is the sustainability strand: [2], [3] make the point, now widely accepted, that compute and energy are first-class costs in any AI deployment. Both strands lead to the same observation we build on: *verbosity is a requirement*. A two-line answer and a forty-line chain-of-thought are different products, with different cost, auditability, and sustainability profiles.

B. Chain-of-Thought and Compressed Reasoning

Wei et al. [4] and Kojima et al. [5] showed that step-by-step prompting substantially improves multi-step reasoning, building on the earlier scratchpad framing of Nye et al. [12]. The cost of those long rationales has motivated work on shortening or distilling them: rationale distillation into smaller students [13], [14], length-controlled CoT variants, and rationale compression. Our SPP condition lives at the extreme of that line: no rationale is emitted at all.

C. Self-Consistency and Search-Based Prompting

Wang et al. [15] sample many CoT trajectories and majority-vote, trading more output for more accuracy; tree-of-thought [16] and graph-of-thought [17] extend that idea with explicit branching. These methods are accuracy-maximising, not cost-minimising. From an RE perspective they sit at the *opposite* end of the verbosity axis from SPP, and we mark them as planned baselines so that the full axis is covered.

D. Pause and Filler Tokens

Goyal et al. [8] train pause tokens directly into the embedding table; Lanham et al. [7] probe whether the verbalised chain is causally responsible for predictions. SPP is a strictly prompt-level, fine-tuning-free analogue of the pause-token idea. Any signal must come from the additional prompt positions and from the instruction itself.

E. Process Supervision and Reasoning Distillation

Lightman et al. [18] train process reward models that score individual reasoning steps. More recent reasoning systems (DeepSeek-R1 [19] and similar) train the entire reasoning trace under RL. These approaches are powerful but expensive, and they live one abstraction level below ours: SPP is what a practitioner can deploy *today* on top of an off-the-shelf model, before any of those training-time interventions are funded.

F. Efficient Inference and Cost-Aware Evaluation

The systems literature on efficient LLM inference covers speculative decoding [20], FlashAttention [21], quantised inference, KV-cache management, and early exit. The cost-aware evaluation literature (HELM [22]) makes the case for reporting cost alongside accuracy as a default. SPP is complementary to all of these: it sits above the inference stack and shrinks the input to it.

TABLE I
TRUSTWORTHY-AI REQUIREMENT CLASSES AFFECTED BY REASONING VERBOSITY. SIGN INDICATES WHETHER SPP, RELATIVE TO FULL COT, MAKES THE REQUIREMENT EASIER (↑) OR HARDER (↓) TO SATISFY.

Requirement	Why verbosity matters	SPP vs. CoT
Cost / budget	token-priced APIs, monthly budgets	↑
Latency / UX	user-perceived response time	↑
Sustainability	compute per request, carbon footprint	↑
Auditability	log size, retention, reviewer load	↑
Transparency	visible rationale for users / regulators	↓
Faithfulness	rationale matching internal computation	∼
Robustness	sensitivity to wording, T, k	∼
Equity of access	per-call cost gating low-budget users	↑

G. Where the Gap Is

What is missing is a requirement-aware empirical comparison. *Cross-model, cross-dataset, prompt-only, no fine-tuning, cost-aware* studies of placeholder-style silent prompting on at least five small open-source models and at least five reasoning benchmarks are, to our knowledge, not in the literature. That is the gap this paper aims to fill, with the explicit goal of producing artefacts a requirements engineer can actually use: numbers per (model, dataset, condition) cell, a frame for how those numbers map onto trustworthy-AI properties, and a checklist for specifying the trade-off.

III. TRUSTWORTHY-AI REQUIREMENTS THAT VERBOSITY TOUCHES

Before we get to the numbers, we want to be concrete about which trustworthy-AI requirements are actually affected when an organisation slides a system along the verbosity axis. Table I sketches a small catalogue. The intent is not to be exhaustive; it is to show that the choice of prompting condition is not a private hyperparameter but an organisational decision that touches multiple stakeholders.

The table is not surprising on its own. What we think is useful is the observation that *the same intervention* (suppressing the visible chain) helps four trustworthy-AI properties and hurts one. That is exactly the kind of trade-off requirements engineering is meant to make explicit. A specification that says "the system shall use chain-of- thought reasoning" silently buys verbosity, latency, energy and audit load that nobody in the room may have agreed to pay for. Conversely, a specification that says "the system shall not verbalise its reasoning" silently gives up the explainability story that compliance teams often expect to be able to show. The empirical question for the rest of the paper is how steep the trade-off actually is.

IV. METHODOLOGY

A. Operational Definition and Scope

We treat SPP as an input–output condition and nothing more. The model sees a prompt with k literal placeholder tokens of the form `<latent_i>` between the question and the final answer slot, together with instructions not to echo them and not to verbalise reasoning. We measure only the final emitted answer and the generated token count. We do not look at activations, attention maps, or hidden states. As a consequence, we do not claim that any latent computation happens in the placeholder positions; any accuracy that SPP recovers is a property of the prompting condition. The placeholder strings are ordinary BPE tokens, not special embeddings.

B. Prompting Conditions

We evaluate five conditions. Figure 1 shows them side by side.

Direct. The model is told to output only the final answer. *Full CoT.* "Think step by step," no length cap. *Short CoT.* CoT with an explicit budget of three short bullets. *SPP.* Placeholder tokens with $k \in \{1, 2, 4, 8, 16\}$, plus the silence instruction. *Hybrid.* Placeholders followed by a mandatory one-sentence visible justification, isolating the effect of any visible rationale on top of the placeholder condition.

The exact templates are released with the source code.

C. Models

We run five open-source instruction-tuned models locally through Ollama: `qwen2.5:0.5b`, `llama3.2:1b`, `qwen2.5:1.5b`, `gemma2:2b`, and `phi3:mini` (3.8B). Models run sequentially on a 16 GB MacBook Air to keep memory pressure bounded. A 7B baseline (`mistral:7b`) was scoped but not run in time for this draft; we mark it as a planned extension.

D. Datasets

We use five reasoning benchmarks, re-exported into a unified JSONL schema with a question, a gold answer, and a normalised answer key: GSM8K [23], SVAMP [24], ARC-Challenge [25], StrategyQA [26], and PrOntoQA [27]. The mix spans grade-school arithmetic, math word problems, scientific multiple choice, open-domain implicit reasoning, and synthetic first-order logic. The choice is intentional: we want the matrix to contain task types where rationales are plausibly load-bearing (arithmetic) and task types where they may not be (yes/no, multiple choice).

E. Evaluation Matrix

The current evaluation sweeps the cartesian product

$$\underbrace{5}_{\text{models}} \times \underbrace{5}_{\text{datasets}} \times \underbrace{5}_{\text{methods}} \times \underbrace{5}_{k} \times \underbrace{2}_{T} \times \underbrace{20}_{N},$$

with N=20 examples per cell at a single fixed seed (42). Methods that do not consume a placeholder budget contribute one k cell rather than five, yielding 13,520 model calls. Figure 2 illustrates the matrix shape and the per-method counts.

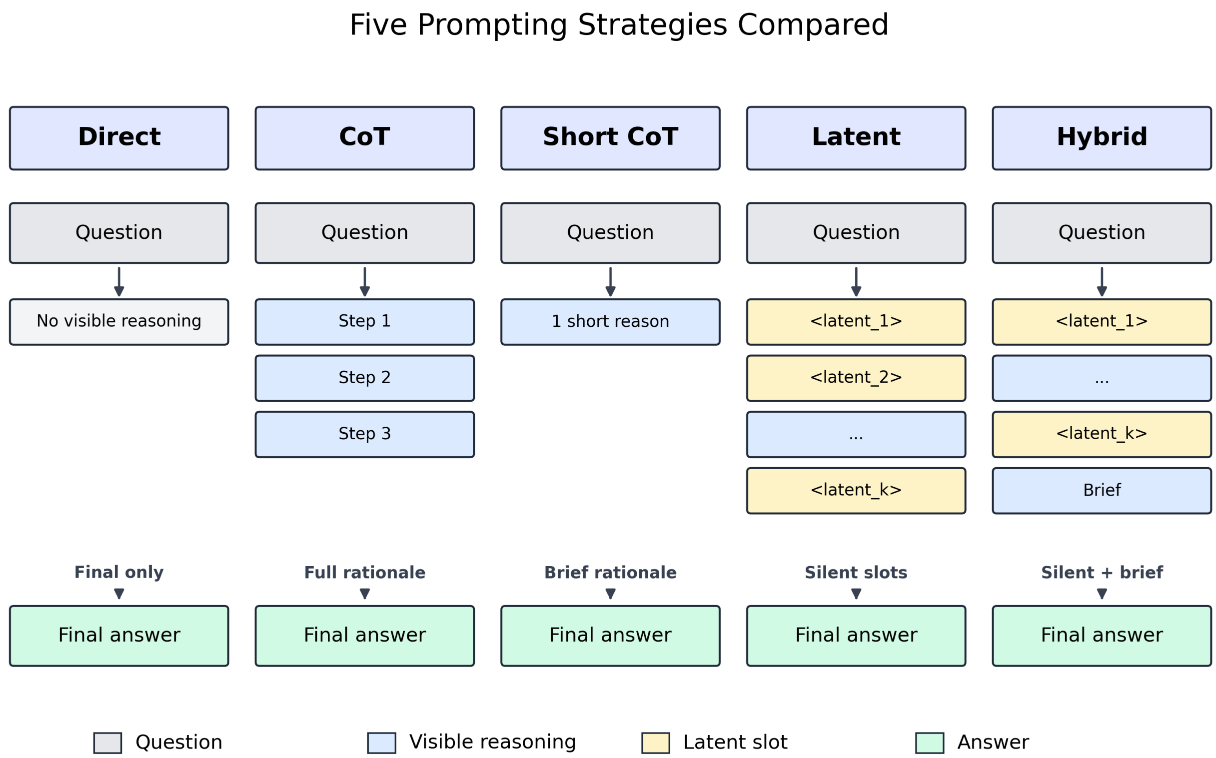

Fig. 1. The five prompting conditions evaluated in this study. For SPP and the hybrid condition, the model is told not to verbalise its reasoning and not to echo the placeholders. For full CoT and short CoT it is told to think step by step (with or without a length cap).

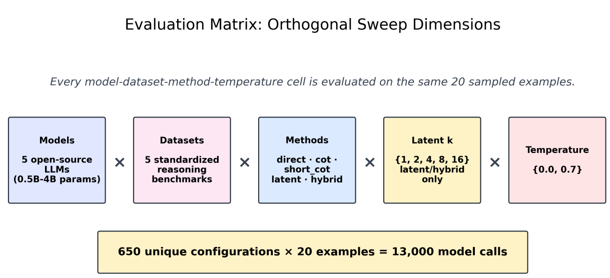

Fig. 2. Shape of the evaluation matrix. SPP and hybrid sweep five placeholder budgets; the three other methods are evaluated once per $(model, dataset)$ cell.

F. Metrics

For each cell we report five quantities. **Accuracy** is exact match against the gold answer after light normalisation (case-folding, punctuation stripping, yes/no canonicalisation). **Output tokens** is the mean tokens generated per response, re-tokenized using the model's own tokenizer. **Latency** is the wall-clock time per call as reported by the inference backend; it conflates server- and client-side overhead and should be read as an indicative cost proxy, not a benchmark. **Accuracy per token** is accuracy divided by mean output tokens. **Tokens per correct answer** is the total tokens emitted in a cell divided by the count of correct answers in that cell.

We do not report confidence intervals or significance tests in this draft. Section IX specifies the planned bootstrap procedure and paired tests, and we are explicit that the headline numbers should be read as point estimates until those numbers land.

G. Implementation

The pipeline is summarised in Figure 3. A matrix orchestrator iterates over $(model, dataset)$ pairs; a per-pair runner iterates over the cells of each pair. Per-example records are written incrementally so that interrupted runs are recoverable. Every record stores the rendered prompt, the raw response, the parsed answer, the gold answer, the correctness label, the output token count, and the latency. Random-seed handling is centralised in a single configuration file; the same configuration file is used to regenerate every figure and every table in this paper.

H. Token Flow Schematic

Figure 4 shows the position of the placeholder tokens in the prompt. The placeholders are ordinary input tokens; the model "sees" them in the same way it sees the question text. Whether they buy any extra capacity for the answer is the empirical question.

V. Baselines

The current evaluation includes the four baselines *Direct*, *Full CoT*, *Short CoT*, and *Hybrid* alongside the SPP condition under study. These are run on the full 13,520-call

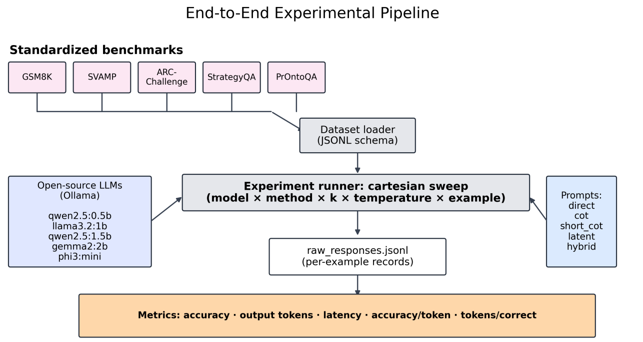

Fig. 3. End-to-end experimental pipeline. All stages are deterministic given the random seed; the current draft uses one seed and is therefore a single point estimate of each cell.

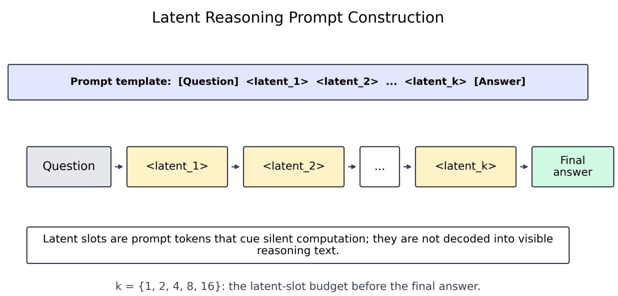

Fig. 4. Placeholder positions in the prompt. The silence instruction is part of the system message; the placeholders are literal substrings in the user message.

matrix and constitute the empirical results in Section VI. We explicitly mark the following stronger baselines as *planned*, with configuration stubs already in the repository under `configs/baselines/`:

- Self Consistency CoT Sample m CoT trajectories and majority-vote [15], with $m \in \{5, 10, 20\}$.
- Least-to-Most Prompting Decompose into ordered subproblems [28], evaluated on GSM8K and StrategyQA.
- Tree-of-Thought style A simple 3-branch variant [16], evaluated on ARC-Challenge and PrOntoQA.
- Compressed-rationale prompting Bounded-token rationale (e.g., $\leq$ 30 tokens) before the answer, building on the rationale- distillation line [13], [14]; the closest published reference point for SPP and our primary missing comparison.

We are deliberately conservative here. The four planned baselines all sit at the *accuracy-maximising* end of the verbosity axis, while SPP, direct, and short CoT sit at the *cost-minimising* end. From an RE perspective, the missing baselines mostly tighten the upper boundary of the trade-off cloud rather than the lower one; SPP's position relative to CoT, the central comparison, is unlikely to flip.

VI. Results

All numbers below are point estimates from a single seed on $N{=}20$ examples per cell. Statistical uncertainty is not yet quantified. We phrase the result claims as suggestive rather than conclusive, and the expanded evaluation plan (Section IX) specifies what comes next. None of the numbers in this section are simulated; they all come out of `paper/results_summary.json` in the released repository.

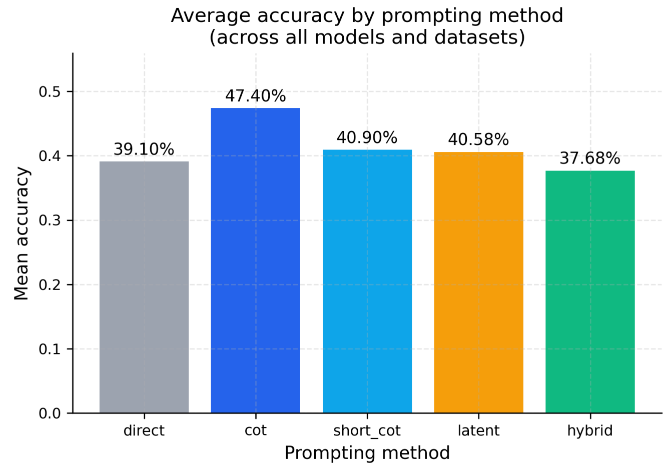

Fig. 5. Mean accuracy by prompting condition, averaged over all 25 $(model, dataset)$ pairs and all placeholder budgets k. Error bars are not yet reported; see Section IX.

TABLE II
POINT ESTIMATES AVERAGED OVER ALL 25 $(model, dataset)$ PAIRS AND ALL PLACEHOLDER BUDGETS. HIGHER IS BETTER FOR ACCURACY AND ACCURACY-PER-TOKEN; LOWER IS BETTER FOR TOKENS AND LATENCY.

Condition	Acc. (%)	Tokens	Lat. (s)	Acc/Tok
Direct	39.1	14.4	0.62	0.0272
Full CoT	**47.4**	236.8	5.95	0.0020
Short CoT	40.9	117.2	2.56	0.0035
SPP (latent)	40.6	15.4	0.67	**0.0263**
Hybrid	37.7	37.3	1.10	0.0101

A. Headline Accuracy and Cost (RQ1)

Table II reports the headline numbers across all 25 $(model, dataset)$ pairs and (where applicable) all five placeholder budgets. Full CoT wins on accuracy, at 47.4%, but it also costs the most: 236.8 tokens per response and 5.95 seconds of latency on the hardware profile used. SPP comes in at 40.6% accuracy — below CoT, but essentially level with short CoT (40.9%) and above the direct baseline (39.1%) — for 15.4 tokens and 0.67 seconds. That is a 93.5% reduction in tokens and an 88.7% reduction in latency relative to CoT, for a 6.8 percentage-point accuracy gap. The hybrid condition lands lower at 37.7% accuracy and 37.3 tokens; a one- sentence visible justification on top of the placeholder buys nothing in this evaluation and costs both accuracy and tokens. We were honestly a little surprised by the hybrid number; we expected the visible-rationale signal to help.

B. Token Cost and Latency

The cost gap is the main story. Figures 6 and 7 show the same picture: full CoT is roughly $15.4\times$ more tokens and

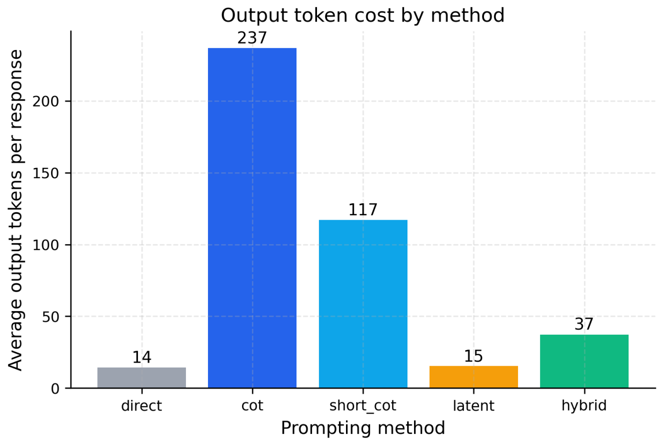

Fig. 6. Mean output tokens per response by prompting condition, averaged across all 25 $(model, dataset)$ pairs.

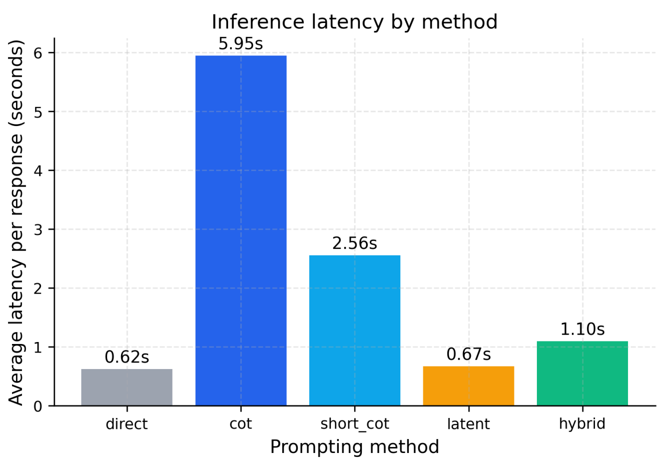

Fig. 7. Mean inference latency per call on the MacBook Air 16 GB profile. Latency is an indicative cost proxy and is sensitive to background load on a consumer machine.

roughly 8.9× more wall-clock time than SPP in our setup. The latency measurement is sensitive to background load on a consumer laptop, so we treat it as an upper- bound estimate of the gap one would see on a server with consistent load — the relative ordering should be similar, but the absolute seconds will be smaller.

C. Cost Effectiveness

When we re-express the trade-off as efficiency ratios, the gap is large. SPP requires roughly 38.0 tokens per correct answer in our sample, against 499.7 for full CoT — a 13.1× difference. Direct is similar to SPP (36.8 tokens per correct), and short CoT and hybrid are intermediate (286.6 and 98.9, respectively). We phrase this as "in our sample" deliberately: the underlying accuracy gap may not survive multi-seed evaluation, but the cost gap is mechanical and is not going to move much. For a requirements engineer staring at a monthly token bill, that distinction matters.

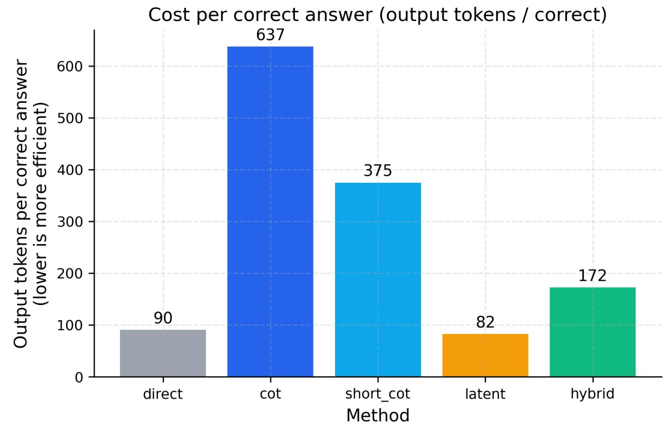

Fig. 8. Tokens generated per correct answer (lower is better).

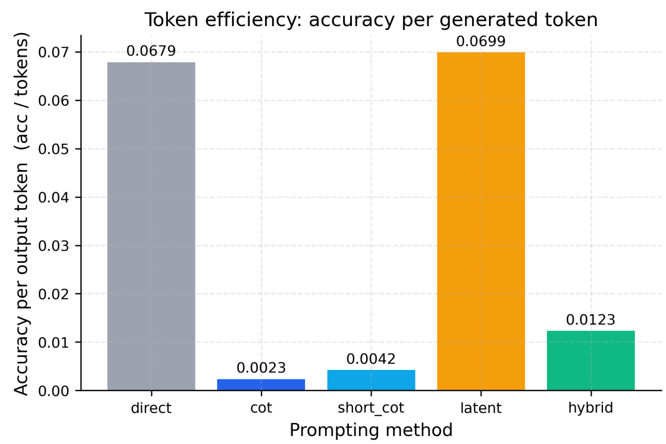

Fig. 9. Accuracy per generated token (higher is better).

D. Accuracy–Token Trade-off

Figure 10 plots every cell in the matrix on the accuracy-vs-tokens plane. Direct and SPP cluster on the cheap end of the token axis; CoT lies on the expensive end; short CoT and hybrid interpolate. The SPP cluster overlaps the lower portion of the CoT cluster on accuracy while sitting an order of magnitude lower on the token axis. We do not claim a strict Pareto improvement: a substantial fraction of the CoT cluster lies above SPP on accuracy. What we do claim is that the token-cost gap is large enough that any RE trade-off analysis that omits it is incomplete.

E. Effect of the Placeholder Budget k

A second result, perhaps the more interesting one for the mechanistic interpretation, is that mean accuracy is essentially flat in k (Fig. 11). Increasing the placeholder budget from 1 to 16 does not move the needle. This is consistent with two readings: the prompting condition itself (the silence instruction) carries the signal, and any extra placeholder positions are not being used for non-trivial hidden computation. We prefer

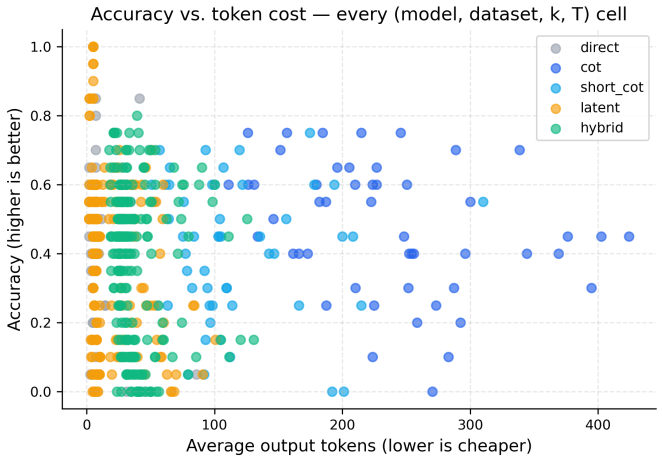

Fig. 10. Per-cell accuracy versus output token cost. Each point is a single $(model, dataset, condition, k, T)$ configuration.

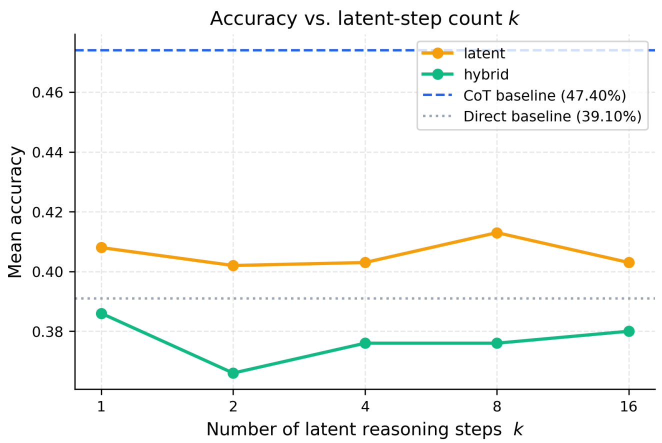

Fig. 11. Accuracy as a function of the placeholder budget k for the SPP and hybrid conditions. Curves are approximately flat in the tested range $k \in \{1, 2, 4, 8, 16\}$.

the first reading because it is the more conservative one and the only one we have evidence for.

F. Per-Dataset Breakdown (RQ2)

The numbers in Table III are the most important in the paper for an RE audience. The mean ranking of conditions is *not* preserved across task types. On GSM8K, full CoT beats SPP by 47 points. On PrOntoQA, SPP beats full CoT by 19 points. The same prompting decision can swing the system from clear-best to clear-worst depending on what the user is asking. That is precisely the kind of behaviour a requirements engineer needs to surface in the specification: a single global choice of prompting condition is underspecified; the right answer routes by task class.

G. Per-Model Sensitivity

Figure 13 shows the per-model picture. The qualitative ranking CoT $\geq$ Short CoT $\approx$ SPP $\geq$ Direct $\geq$ Hybrid holds in mean across all five evaluated models. Quantifying ranking

TABLE III
PER-DATASET ACCURACY (%) AT T=0. SPP LOSES HEAVILY ON ARITHMETIC WORD PROBLEMS (GSM8K, SVAMP) AND MATCHES OR BEATS COT ON CLASSIFICATION-STYLE BENCHMARKS (ARC-C, STRATEGYQA, PRONTOQA).

Dataset	Direct	CoT	Short CoT	SPP	Hybrid
GSM8K	10.0	**58.0**	36.0	10.8	12.0
SVAMP	32.0	**61.0**	54.0	34.0	37.8
ARC-C	38.0	26.0	17.0	**46.0**	31.2
StrategyQA	56.0	55.0	58.0	50.4	47.0
PrOntoQA	66.0	49.0	51.0	**67.6**	62.2

TABLE IV
EFFECT OF THE PLACEHOLDER BUDGET k ON SPP AND HYBRID AT T=0. ACCURACY IS APPROXIMATELY FLAT IN k.

k	1	2	4	8	16
SPP acc. (%)	42.4	41.8	41.8	41.6	41.2
SPP tok.	12.8	12.3	11.2	11.6	11.6
Hybrid acc. (%)	39.6	37.8	38.4	38.0	36.4
Hybrid tok.	32.0	32.3	31.5	31.7	31.4

stability with Kendall τ or Spearman correlation is on the plan once multi-seed data is available.

H. Heat Map

Figure 14 averages over conditions and shows the $(model, dataset)$ accuracy surface; this is the easiest summary to pass to a non-ML colleague who wants to understand which combinations are usable at all.

I. Robustness to Decoding Temperature

Figure 15 shows the ranking is preserved across T=0.0 and T=0.7 in our sample. Robustness to prompt wording is not yet evaluated; we plan a paraphrase sweep with at least three wording variants per condition.

J. Effect of k, Numerically

Table IV reports the SPP and hybrid numbers as a function of k at T=0. Accuracy moves by about a percentage point across the tested range; mean tokens move by roughly one and a half.

VII. ANALYSIS AND FAILURE MODES

A. When Does Suppressing the Rationale Hurt?

The clearest pattern in the data is that suppressing the rationale hurts on tasks where intermediate numerical state is load-bearing, and does not hurt on tasks where the answer is one of a small symbolic set. GSM8K and SVAMP both require carrying a number through several steps; if the model is not allowed to write that number down, it has to keep it in a single forward pass, and small models are not very good at that. ARC-C, StrategyQA, and PrOntoQA all collapse to a short answer space (one of four options, yes/no, true/false). On those, the cost of writing out a rationale outweighs the benefit, at least at the model scales we evaluate, and SPP performs as well as or better than CoT.

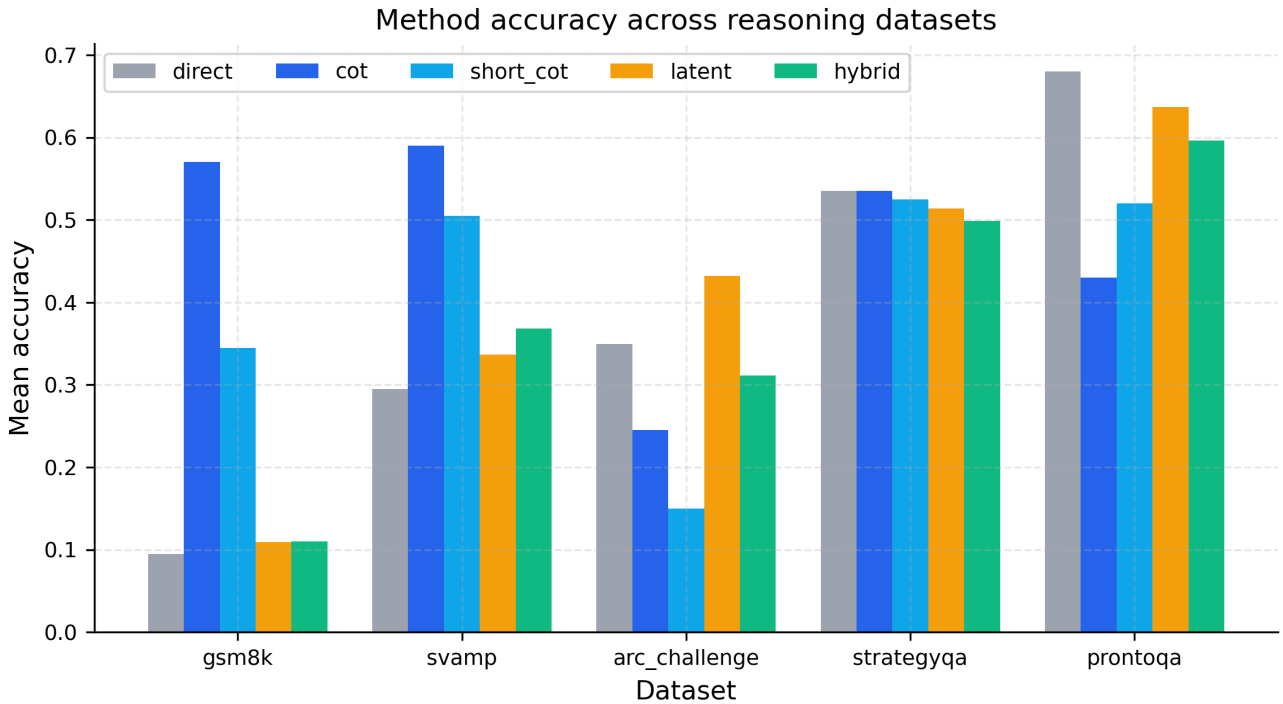

Fig. 12. Per-dataset accuracy by prompting condition. The task-by-condition interaction is the strongest single effect we observe, and it is the one most relevant to a requirement engineer considering routing rules.

B. Failure Mode Taxonomy

We outline a taxonomy derived from informal inspection of a non-random subset of responses. A blind annotation pass over a stratified sample of ≥ 100 responses per condition is on the plan; these categories should be read as a hypothesis to test, not as a measured frequency table.

1) **Arithmetic slips.** A numeric answer one step away from the gold value. Most common on GSM8K and SVAMP under SPP.
2) **Symbolic-manipulation errors.** A logical operator or quantifier mishandled. Observed on PrOntoQA in a non-trivial minority of cases.
3) **Premature commitment.** The model picks one of the answer options without engaging with the question. Most visible on ARC-C under SPP and direct.
4) **Instruction-following failure.** The model emits an unstructured response without a `Final Answer:` line; the evaluator's last-line fallback usually still extracts the answer.
5) **Placeholder over-compression.** Under hybrid, the one-sentence justification leaks more than the format allows.
6) **Verbosity leakage.** Under SPP, the model occasionally emits a short visible reasoning fragment despite the silence instruction. Rare but model-specific.
7) **Format / parsing failure.** The answer is correct but in an unexpected format (e.g., a fraction instead of a decimal), and the strict matcher rejects it.

For an RE audience, categories 4 and 7 are particularly important: they are not really model failures, they are specification failures. The system did its job; the contract was wrong.

C. Specifying SPP and CoT as RE Acceptance Criteria

We close the analysis with a small sketch of how a practitioner might write down acceptance criteria for the two ends of the verbosity axis, in a form a reviewer could check. This is intentionally minimal, to make a point about granularity, not to be a finished framework.

- **R-cost.** Mean output tokens per response on the user-traffic distribution shall not exceed τ_{tok}. (Pick τ_{tok} from a token budget; SPP fits easily under any $\tau_{\text{tok}} \geq 32$ in our sample, full CoT does not.)
- **R-lat.** P95 latency per call shall not exceed τ_{lat} on the deployment hardware profile. (CoT will blow this on consumer hardware in our setup; SPP will not.)
- **R-acc-class.** For request class c, accuracy shall not fall below α_c. (Different α_c for arithmetic vs. classification puts the routing rule in the requirement document itself.)
- **R-trans.** If the request class is regulated, a visible rationale shall be available to the auditor; otherwise it may be suppressed. (This is the explicit transparency–cost trade-off.)
- **R-faith.** Where a visible rationale is mandatory, a faithfulness probe shall be run on a sample of responses [6], [7]. (The rationale being visible does not entail it being honest.)

The one we expect to be most contentious in practice is R-trans. A common reflex in compliance discussions is to

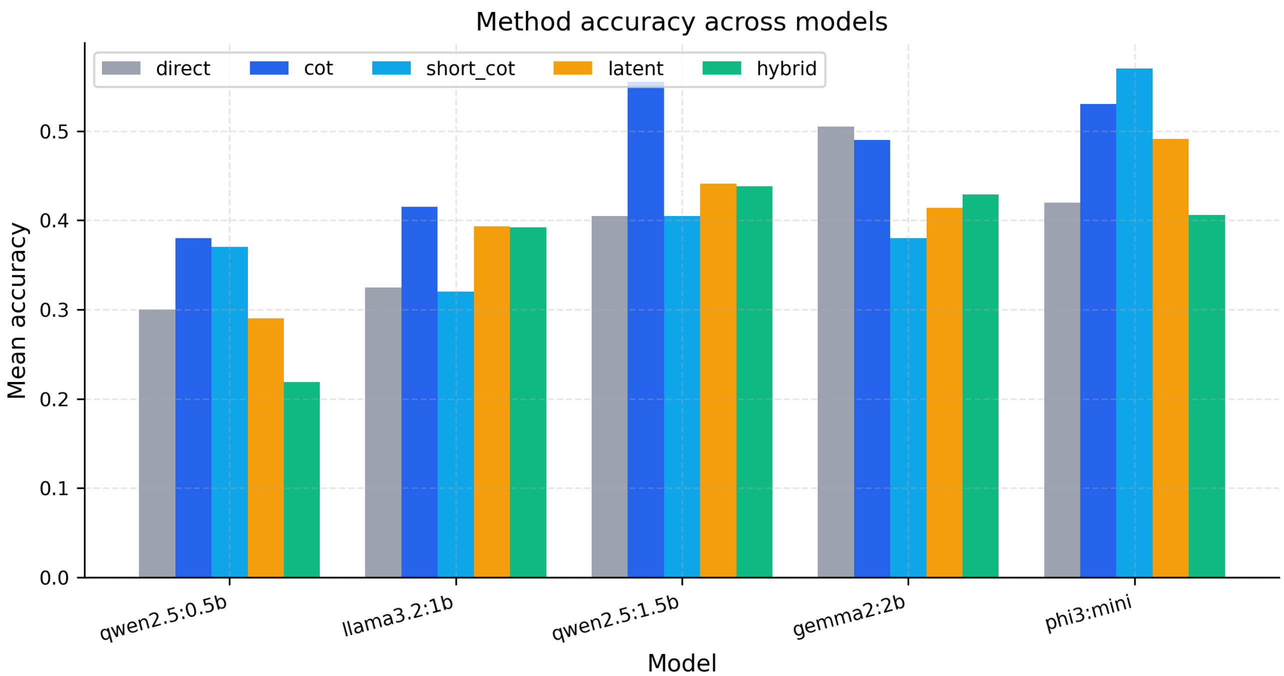

Fig. 13. Per-model accuracy by prompting condition. Ranking is largely preserved across models.

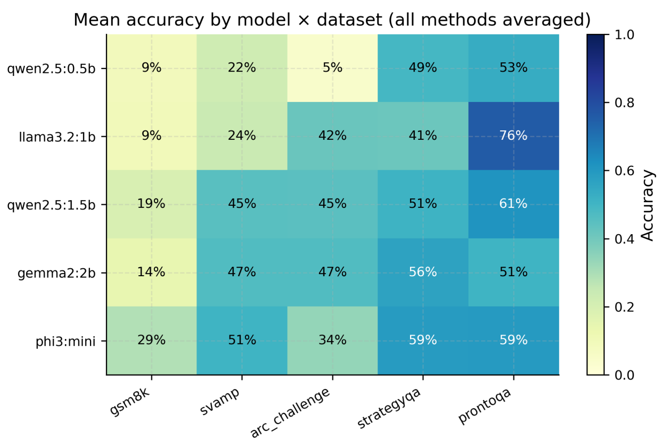

Fig. 14. Mean accuracy heat map across models and datasets, averaged over all prompting conditions and placeholder budgets.

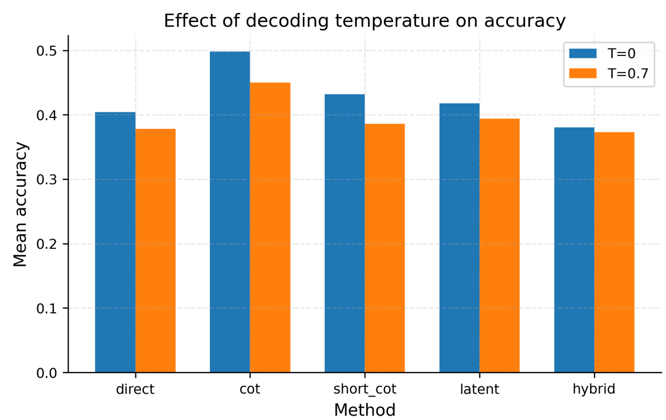

Fig. 15. Effect of decoding temperature on accuracy by prompting condition. The ranking is preserved at both temperatures evaluated in this sample.

demand a visible rationale for everything. The faithfulness literature [6], [7] should make us cautious about that reflex; a long, plausible-sounding chain is not the same as an honest one. SPP, on regulated request classes, is the wrong default. On unregulated, cost-bound request classes, demanding it is expensive theatre.

VIII. Threats to Validity

We follow the standard internal/external/construct framing.

Internal. A single random seed; $N{=}20$ per cell; one prompt template per condition; latency measured on a consumer laptop. Together these mean the headline numbers are point estimates and should be treated as such.

External. Five small open-source models (0.5B–3.8B). Larger models may behave differently; we have no $\geq$ 7B data point in this evaluation. Five reasoning benchmarks; the picture on programming, clinical, or legal reasoning is not characterised here.

Construct. Output tokens are a proxy for cost. The mapping to monetary cost depends on the API used; the mapping to energy depends on the inference stack. Latency is measured on consumer hardware with non-zero background load. “Accuracy” is exact match after light normalisation, which undercounts cases where the answer is correct but in an unexpected format.

Conclusion. We do not run paired significance tests in this draft, so claims about "SPP $\approx$ short CoT" should be read as "equal up to the precision of a $N{=}20$, single-seed estimate". Section IX specifies the gating tests for stronger language.

IX. Expanded Evaluation Plan

We document the planned extensions explicitly so that this draft can be cleanly upgraded to a longer version.

Larger samples. Increase N from 20 to at least 200 per dataset, or to the full standard split where feasible (GSM8K test 1,319; SVAMP test 1,000; ARC-C test 1,172; StrategyQA dev 229; PrOntoQA stratified to 200–500). *Multiple random seeds.* At least three; five preferred. *Bootstrap CIs.* 95% percentile intervals on accuracy, tokens, latency, accuracy-per- token, and tokens-per-correct, using 1,000 resamples. *Paired significance.* For every $(dataset, model)$ pair, paired bootstrap or McNemar tests on item-level correctness between SPP and each baseline, with Benjamini–Hochberg correction across the matrix. *Stronger baselines.* Self-consistency (m $\in \{5, 10, 20\}$), least-to-most, a 3-branch tree-of-thought variant, and compressed-rationale prompting. *Prompt-paraphrase robustness.* At least three wording variants per condition; report variance across paraphrases. *Larger model.* Run the matrix on a 7B baseline. *Contamination probe.* Where training corpora are documented, sample-level overlap checks on the five benchmarks.

X. Implications for RETRAI Practice

We close on what we hope is the more useful contribution: not the numbers themselves but the discipline of writing them down. Three points stand out for a RETRAI audience.

First, *verbosity is a requirement, not an implementation detail*. The choice between SPP and full CoT swings token cost by an order of magnitude and accuracy by 6.8 points on average (and by much more on individual task classes). Specifications that do not mention prompting policy are buying that swing implicitly.

Second, *the right policy is conditional on the request class*. The per-dataset table is not just an interesting empirical fact; it is a requirement-engineering instruction. Routing arithmetic-style queries to CoT and classification-style queries to SPP recovers most of the cost savings without paying the arithmetic penalty. An RE process should ask, for every system, whether it is feasible to write down a class-conditional verbosity policy of this kind.

Third, *transparency and faithfulness are different things*. The compliance reflex of always demanding a visible rationale is in some tension with the faithfulness literature [6], [7]. SPP is the obvious right default for unregulated, cost-bound request classes; for regulated classes, a visible rationale is necessary but not sufficient, and an explicit faithfulness probe should be part of the acceptance criteria.

XI. Conclusion

We treated reasoning verbosity as a trustworthy-AI requirement and characterised one cost-end option for it, silent placeholder prompting, on a 13,520-call evaluation matrix over five small open-source LLMs and five reasoning benchmarks. SPP cuts mean output tokens by 93.5% relative to full CoT, at the price of 6.8 percentage points of mean accuracy, with a strong task-conditional twist: on classification-style benchmarks SPP matches or beats CoT, and on arithmetic word problems it does not. We sketched a small catalogue of trustworthy-AI requirements that the verbosity choice touches and a five-item acceptance-criteria checklist a practitioner could write down. The contribution is empirical and methodological; we make no mechanistic claim, and we are explicit about the preliminary statistical status of the headline numbers. Code, prompts, raw records, configurations, and figures are released so that the matrix can be re-run, extended, or instrumented with stronger baselines.

Reproducibility Statement

All code, prompts, downloaded datasets, raw per-example records, plotting scripts, and the figures used in this manuscript are released with the project. A single configuration file pins the random seed, the model identifiers, the dataset paths, and the full hyperparameter grid; running it regenerates every number reported here. The expanded evaluation plan in Section IX is released as a set of configuration stubs under `configs/baselines/` so that an upgrade run reproduces the same schema.

Acknowledgements

We thank the open-source community for the model weights and benchmark releases on which this study depends.

References

[1] European Parliament and Council, "Regulation (eu) 2024/1689 of the european parliament and of the council laying down harmonised rules on artificial intelligence (artificial intelligence act)," 2024, official Journal of the European Union, L series, 12 July 2024.

[2] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[3] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, 2020.

[4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, 2022.

[5] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems*, 2022.

[6] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in *Advances in Neural Information Processing Systems*, 2023.

[7] T. Lanham, A. Chen, A. Radhakrishnan, B. Steiner, C. Denison, D. Hernandez, D. Li, E. Durmus, E. Hubinger, J. Kernion *et al.*, "Measuring faithfulness in chain-of-thought reasoning," *arXiv preprint arXiv:2307.13702*, 2023.

[8] S. Goyal, Z. Ji, A. S. Rawat, A. K. Menon, S. Kumar, and V. Nagarajan, "Think before you speak: Training language models with pause tokens," in *International Conference on Learning Representations*, 2024.

[9] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein *et al.*, "On the power of foundation models," in *arXiv:2108.07258*, 2021.

[10] G. Vilone and L. Longo, "Explainable artificial intelligence: a systematic review," in *arXiv:2006.00093*, 2020.

[11] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," NIST AI 100-1, Tech. Rep., 2023.

[12] M. Nye, A. J. Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, C. Sutton, and A. Odena, "Show your work: Scratchpads for intermediate computation with language models," in *NeurIPS Workshop on Deep Learning Through Information Geometry*, 2021.

[13] L. C. Magister, J. Mallinson, J. Adamek, E. Malmi, and A. Severyn, "Teaching small language models to reason," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

[14] N. Ho, L. Schmid, and S.-Y. Yun, "Large language models are reasoning teachers," *arXiv preprint arXiv:2212.10071*, 2022.

[15] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations*, 2023.

[16] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems*, 2023.

[17] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of thoughts: Solving elaborate problems with large language models," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[18] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *International Conference on Learning Representations*, 2024.

[19] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[20] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*, 2023.

[21] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, 2022.

[22] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar *et al.*, "Holistic evaluation of language models," *arXiv preprint arXiv:2211.09110*, 2022.

[23] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[24] A. Patel, S. Bhattamishra, and N. Goyal, "Are nlp models really able to solve simple math word problems?" in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021.

[25] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? try arc, the ai2 reasoning challenge," *arXiv preprint arXiv:1803.05457*, 2018.

[26] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, and J. Berant, "Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies," *Transactions of the Association for Computational Linguistics*, vol. 9, 2021.

[27] A. Saparov and H. He, "Language models are greedy reasoners: A systematic formal analysis of chain-of-thought," in *International Conference on Learning Representations*, 2023.

[28] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. V. Le, and E. H. Chi, "Least-to-most prompting enables complex reasoning in large language models," in *International Conference on Learning Representations*, 2023.